1. FUNDAMENTOS DE ESTRUCTURAS DE DATOS DINÁMICAS

Definición Técnica de Listas

Una lista en Python es una estructura de datos heterogénea, mutable y secuencial. A diferencia de los arreglos estáticos, las listas en Python actúan como arreglos dinámicos, permitiendo la reasignación de memoria y el crecimiento de la estructura en tiempo de ejecución.

Atributos de Ingeniería:

- Indexación Zero-Based: El acceso a los nodos se realiza mediante un índice entero, donde el primer elemento ocupa la posición 0.
- Mutabilidad: Capacidad de modificar el contenido de la estructura sin alterar su identidad (ID de memoria/Puntero).
- Complejidad Algorítmica: El acceso por índice es de tiempo constante $O(1)$, mientras que la búsqueda por valor es de tiempo lineal $O(n)$.

2. PROTOCOLO DE MÉTODOS Y OPERACIONES SOBRE LA ESTRUCTURA

Para gestionar eficientemente las colecciones, el ingeniero debe dominar los siguientes métodos de la clase list:

1. Inserción de Datos:
    - .append(obj): Inserción al final de la secuencia.
    - .insert(index, obj): Inserción por desplazamiento (desplaza los índices posteriores).
2. Eliminación y Depuración:
    - .pop(index): Eliminación por índice. Es el método estándar para manejo de pilas y colas.

- .remove(value): Eliminación por coincidencia de valor. Requiere validación previa para evitar excepciones.

3. Transformación y Ordenamiento:
    - .sort(): Implementación del algoritmo Timsort para organizar la estructura In-place.
    - .reverse(): Invierte el orden de los elementos actuales.

3. ARQUITECTURA DE VALIDACIÓN Y SANITIZACIÓN

Antes de que un dato sea persistido en una lista, debe atravesar una capa de Sanitización de Entradas. Esto garantiza la integridad de la información y evita registros redundantes o ruidosos (espacios accidentales, inconsistencia en mayúsculas/minúsculas).

- Ver Script 1: Implementación de validación de unicidad y limpieza de cadenas.

4. LABORATORIO DE APLICACIÓN (ENTREGABLES)

PASO 1: Diagnóstico de Comportamiento de Memoria

El estudiante debe codificar un script de prueba donde demuestre el principio de mutabilidad.

- Ver Script 2: Prueba de ID de memoria y manipulación secuencial.

PASO 2: Reingeniería del MVP (Avance 6 del Proyecto)

Debe evolucionar la lógica de su proyecto central bajo los siguientes lineamientos:

- Sustitución de Persistencia: Elimine el uso de variables escalares individuales y centralice la información en colecciones indexadas.

- Capa CRUD: Implemente funciones modulares para la inserción (Create) y visualización (Read) de los datos.
- Formateo de Salida: Utilice el método enumerate() para generar listados técnicos numerados y bilingües.
- Ver Script 3: Modelo de integración CRUD para el sistema de tránsito.

5. GESTIÓN DE CONFIGURACIÓN Y VERSIONES (GIT)

Se exige el cumplimiento del estándar de mensajes descriptivos para el control de versiones:

- Comando Sugerido: git commit -m "feat: implementar almacenamiento dinámico y lógica de sanitización en colecciones